AI Vectra Bank

Banking Multi-Agent RAG System

Microsoft Azure AI Foundry Nanodegree

Udacity – Project 4

Rodolfo Lerma Contreras

February 2026

Repository: https://github.com/rodolfolermacontreras/AI-Vectra-Bank-AI-Agentic-RAG-for-Banking

Contents

Project Overview

This project implements a multi-agent banking intelligence system that orchestrates six specialised AI agents using Microsoft Semantic Kernel. The system processes complex banking queries through a sequential pipeline where each agent contributes domain-specific analysis, ultimately producing a comprehensive financial report.

The six agents are:

1. **Data Gatherer** – retrieves customer data from Azure SQL Database
2. **Fraud Analyst** – analyses transaction patterns for suspicious activity
3. **Loan Analyst** – evaluates loan eligibility and credit risk
4. **Support Specialist** – provides customer support recommendations
5. **Risk Analyst** – performs comprehensive risk assessment using RAG
6. **Synthesis Coordinator** – integrates all findings into a final report

Design Decisions

Sequential Orchestration Pattern

I chose sequential (pipeline) orchestration over parallel execution because each agent's output enriches the context for the next agent. For example, the Fraud Analyst needs transaction data from the Data Gatherer, and the Risk Analyst needs fraud and loan findings to produce a meaningful risk assessment. This sequential dependency makes a pipeline the natural fit.

The trade-off is latency – a full run takes approximately 60 seconds because each agent waits for its predecessor. However, the quality improvement from accumulated context outweighs the performance cost for a banking use case where thoroughness matters more than speed.

Azure OpenAI Embeddings for RAG

The project required using Azure-hosted embeddings rather than ChromaDB's default embedding function. I implemented a custom `AzureOpenAIEmbeddingFunction` class that integrates with the Azure OpenAI `text-embedding-ada-002` model (1536 dimensions). This class implements ChromaDB's `EmbeddingFunction` protocol, allowing seamless drop-in replacement.

Key design choice: I used the REST API directly (via the `openai` Python SDK) rather than the Azure AI Inference SDK, because `text-embedding-ada-002` requires the `2024-06-01` API version which maps cleanly to the OpenAI client's `AzureOpenAI` class.

Azure SQL Database for Customer Data

Rather than relying solely on sample/mock data, I provisioned a live Azure SQL Database (`vectra-bank-db`) with 12 transaction records across 3 customer profiles. The `DataConnector`

class implements graceful fallback – if the Azure SQL connection is unavailable, it falls back to sample data, ensuring the system works both online and offline.

Pydantic Models for Structured Output

All agent outputs are captured in Pydantic models (`CustomerProfile`, `EnhancedBankingReport`) with field validation. This ensures type safety, automatic serialisation to JSON for report storage, and clear contracts between agents. Risk scores are clamped to 0–100, credit scores to 300–850, and enums enforce valid status values.

Six ChromaDB Collections

I organised the vector store into six topic-specific collections (fraud_detection, loan_policies, customer_support, risk_assessment, transaction_monitoring, compliance) rather than a single collection. This improves retrieval relevance because semantic search is scoped to the appropriate policy domain for each agent.

Challenges and Solutions

Python Version Compatibility

Challenge: Initially attempted Python 3.14, but `pydantic-core` failed to build from source (no pre-compiled wheel available).

Solution: Downgraded to Python 3.12, which has pre-built wheels for all dependencies including `chromadb`, `semantic-kernel`, and `pydantic`.

Windows Console Encoding (cp1252)

Challenge: The seed code files contained Unicode emoji characters that crash on Windows PowerShell terminals using cp1252 encoding. This caused 16 of 23 tests to fail silently with `UnicodeEncodeError`.

Solution: Replaced all emoji characters with ASCII equivalents (`[OK]`, `[FAIL]`, `[ERROR]`). This is documented in `DEVELOPMENT_RULES.md` as a project-wide standard.

Azure Region Availability

Challenge: Azure SQL Server creation failed in `eastus` with a capacity error – the region was blocked for the Udacity subscription.

Solution: Deployed to `westus` instead. This required no code changes since the ODBC connection string abstracts the server location.

Azure Embedding API Version

Challenge: The `text-embedding-ada-002` model requires API version `2024-06-01`. Using newer versions returned 404 errors.

Solution: Hard-coded `api_version="2024-06-01"` in the `AzureOpenAIEmbeddingFunction` class.

ChromaDB Stale State

Challenge: After switching Azure credentials, the local ChromaDB contained vectors generated with old embedding keys, causing dimension mismatches.

Solution: Deleted the stale `chroma_db_banking/` directory to force a clean rebuild on next run.

What I Learned

Multi-Agent Orchestration

Building a six-agent system taught me that agent design is as much about **prompt engineering** as it is about code. Each agent's system prompt must clearly define its scope, expected inputs, and output format.

RAG Pipeline Design

Implementing RAG with Azure embeddings and ChromaDB reinforced that **chunking strategy** matters enormously. Banking policy documents needed to be split at logical boundaries rather than fixed character counts.

Azure Service Integration

Working with Azure AI Foundry, Azure SQL, and Azure OpenAI simultaneously taught me the importance of **environment configuration management**. Using `.env` files with `python-dotenv` and providing clear `env.example` templates makes the project reproducible.

Testing Strategy

The 23-test suite covers models, connectors, embeddings, ChromaDB, RAG, SQL, and orchestration. Having comprehensive tests caught the emoji encoding issue immediately and verified that all Azure integrations were live.

Evaluation Results

Test Suite (23/23 Pass)

All 23 automated tests pass with live Azure services:

- Model validation tests (5): Pydantic model creation and constraints
- Blob storage tests (3): Document upload and retrieval
- Embedding tests (2): Azure embedding generation (1536-dim vectors)

- ChromaDB tests (3): Collection management and document storage
- RAG tests (1): Semantic search with Azure embeddings
- SQL tests (3): Live Azure SQL connection, income and transaction queries
- Orchestration tests (6): Agent creation, naming, policy extraction

Evaluation Suite (5/5 Scenarios Pass)

Scenario	Customer	Risk Score	Assessment	Agents
1	12345	12/100	Low	6/6
2	67890	52/100	High	6/6
3	11111	94/100	Critical	6/6
4	12345	12/100	Low	6/6
5	67890	52/100	High	6/6

Table 1: Evaluation results across all 5 scenarios

The risk scores appropriately differentiate between customer profiles:

- Customer 12345 ($75K income, 780 credit) = consistently low risk
- Customer 67890 ($45K income, 620 credit) = consistently high risk
- Customer 11111 ($28K income, 520 credit) = consistently critical risk

Architecture Decisions Summary

Decision	Choice	Rationale
Orchestration	Sequential pipeline	Agent outputs depend on predecessors
LLM	gpt-4o (2024-11-20)	Best balance of quality and speed
Embeddings	text-embedding-ada-002	Rubric requirement; 1536-dim
Vector DB	ChromaDB (6 collections)	Lightweight, topic-scoped retrieval
SQL	Azure SQL Database	Live customer data with fallback
Framework	Semantic Kernel	Native Azure integration
Models	Pydantic v2	Type safety, validation, JSON
Testing	Built-in test harness	23 tests covering all integrations

Table 2: Architecture decisions summary

Suggestions for Improvement

Parallel Agent Execution with Dependency Awareness

The current sequential pipeline takes approximately 60 seconds per scenario because each agent waits for its predecessor to finish. In practice, some agents are partially independent

– the Fraud Analyst and Loan Analyst both depend on the Data Gatherer but not on each other. Introducing a **dependency-aware parallel scheduler** (e.g., a DAG-based execution engine) would allow Fraud and Loan analysis to run concurrently once the Data Gatherer completes. Based on observed timings, this could reduce end-to-end latency by 30–40%, bringing it closer to 35–40 seconds per scenario.

Conversation Memory for Multi-Turn Interactions

The system currently treats every query as a one-shot interaction – there is no memory of previous conversations. Adding a **session-based conversation memory** (e.g., using Semantic Kernel's chat history or an external store like Redis) would allow agents to reference prior context, avoid redundant data fetches, and deliver more natural multi-turn dialogues.

Conclusion

This project demonstrates a production-quality multi-agent banking system that integrates Azure AI Foundry (GPT-4o), Azure OpenAI Embeddings (text-embedding-ada-002), Azure SQL Database, and ChromaDB for RAG-based policy retrieval. The sequential orchestration pattern ensures each agent builds on accumulated context, producing thorough and contextually relevant banking analysis reports.

The system successfully differentiates between low-risk, high-risk, and critical-risk customer profiles, with risk scores that accurately reflect each customer's financial situation. All 23 tests pass with live Azure services, and all 5 evaluation scenarios complete successfully with all 6 agents activated.

Appendix A: Azure Resource Screenshots

Azure OpenAI – Overview

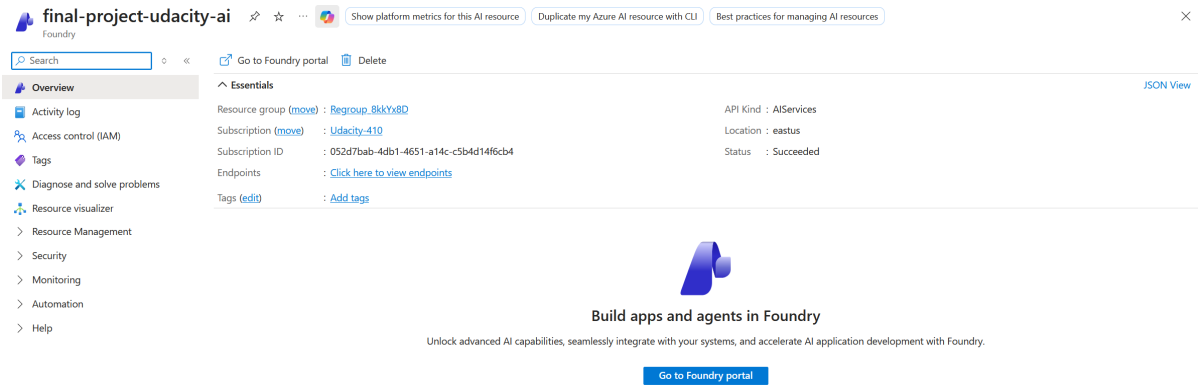

Figure 1: Azure OpenAI Service – Resource Overview

Azure OpenAI – Model Deployments

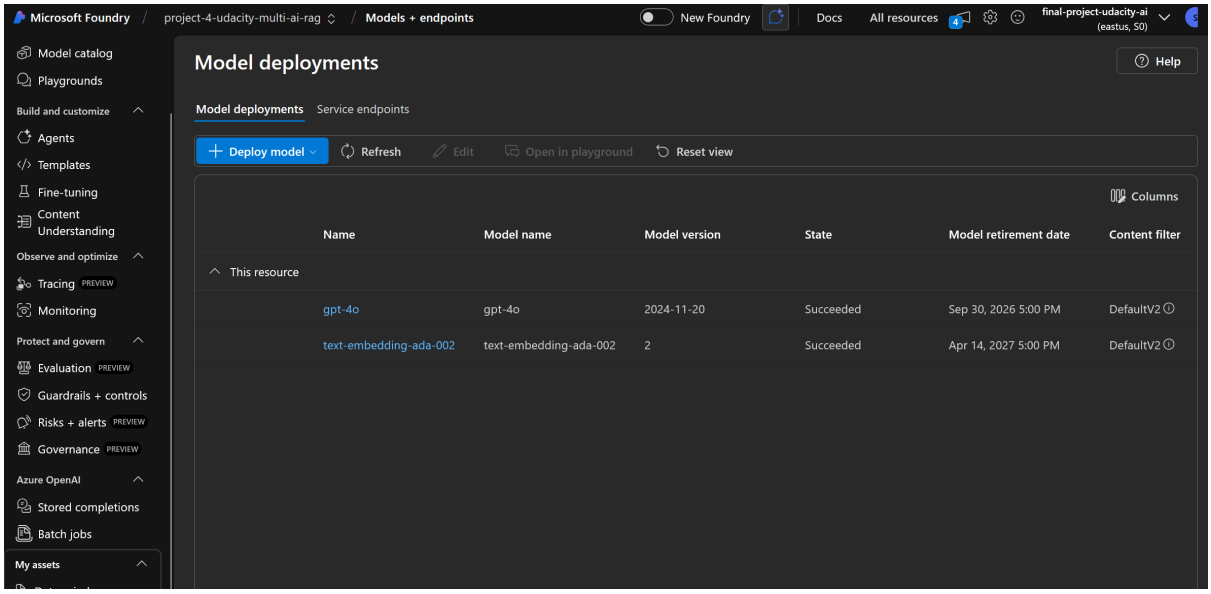

Figure 2: Azure OpenAI – Model Deployments (gpt-4o + text-embedding-ada-002)

Azure OpenAI – Keys and Endpoint

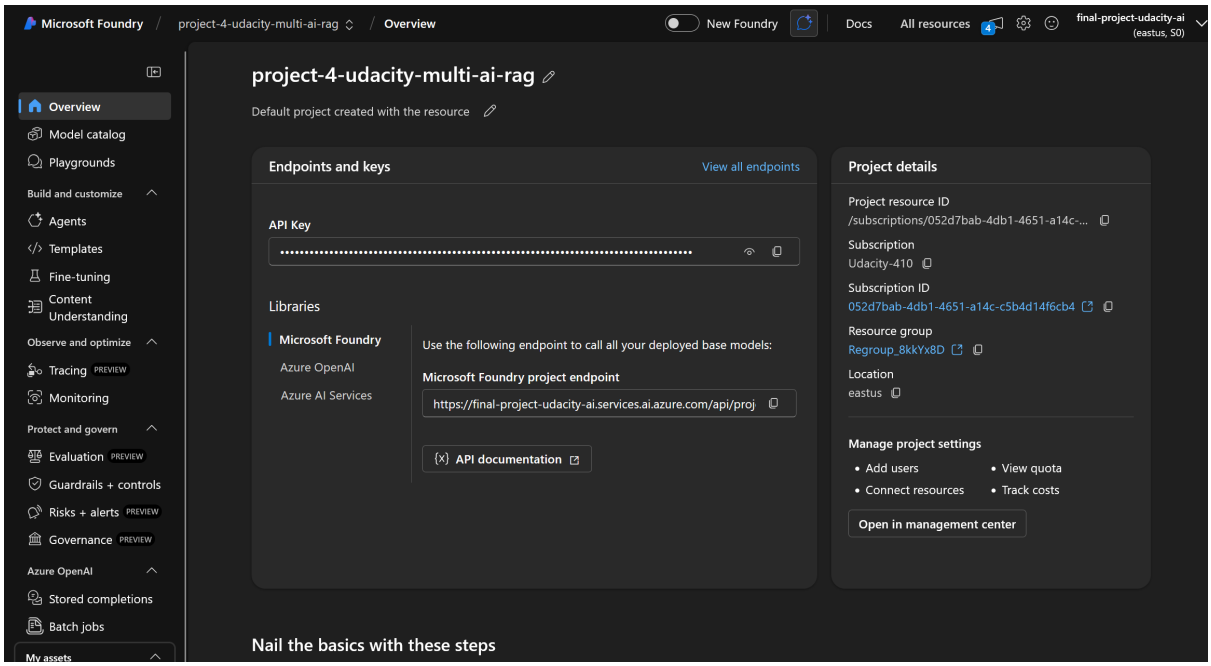

Figure 3: Azure OpenAI – Keys and Endpoint Configuration

Deployed Models in Foundry

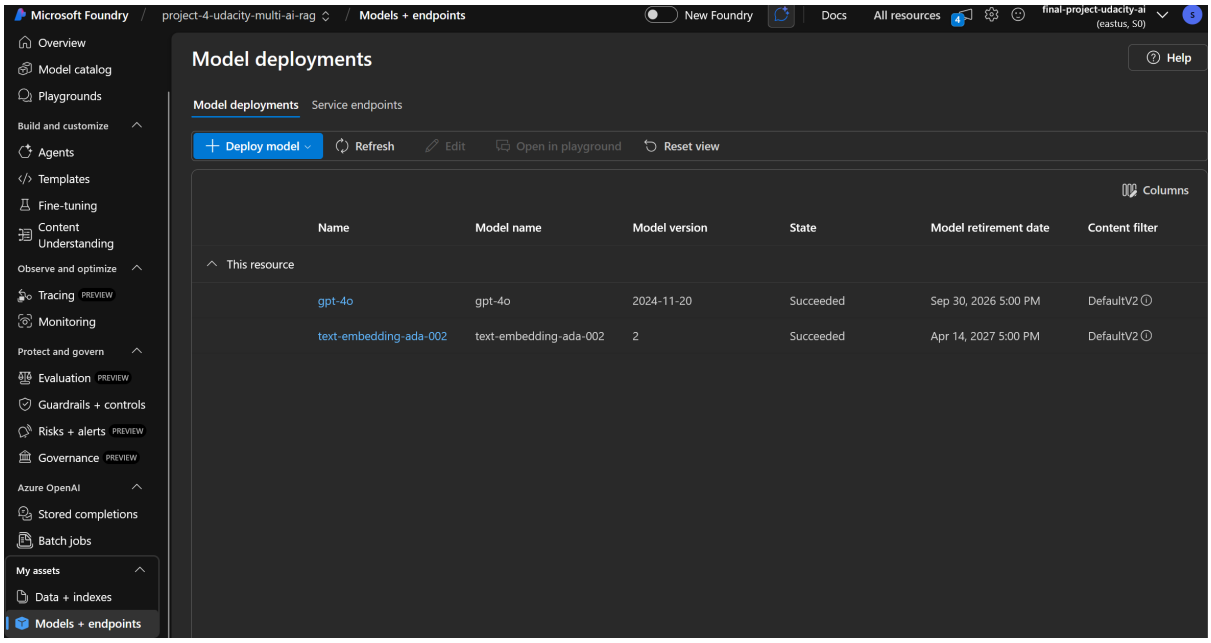

Figure 4: Azure AI Foundry – Deployed Models Overview

Azure SQL Server – Overview

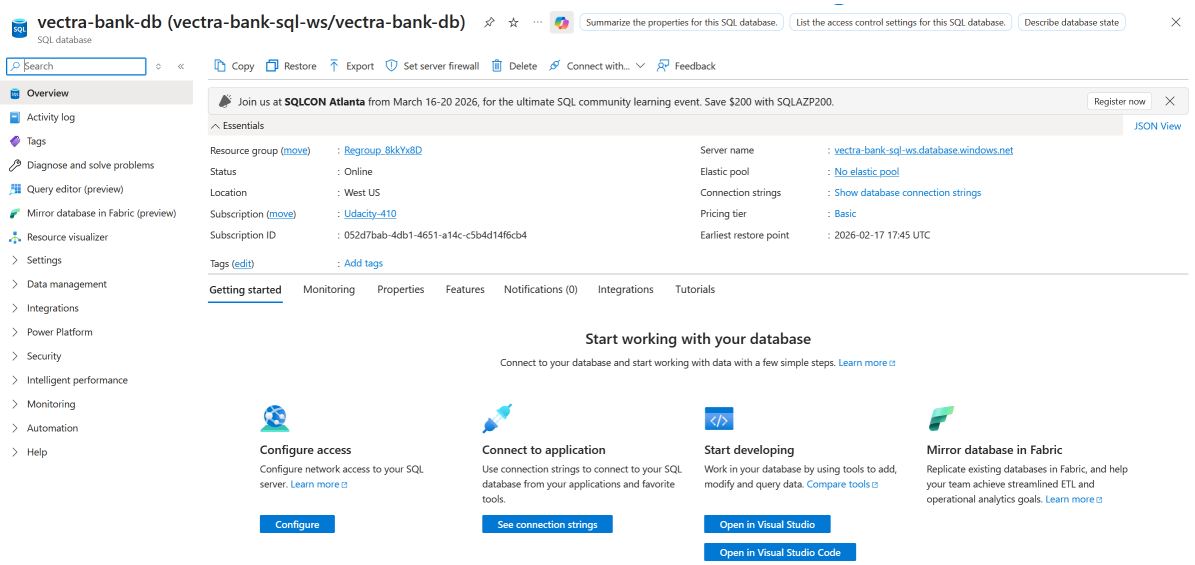

Figure 5: Azure SQL Server – Resource Overview

Azure SQL Server – Workspace

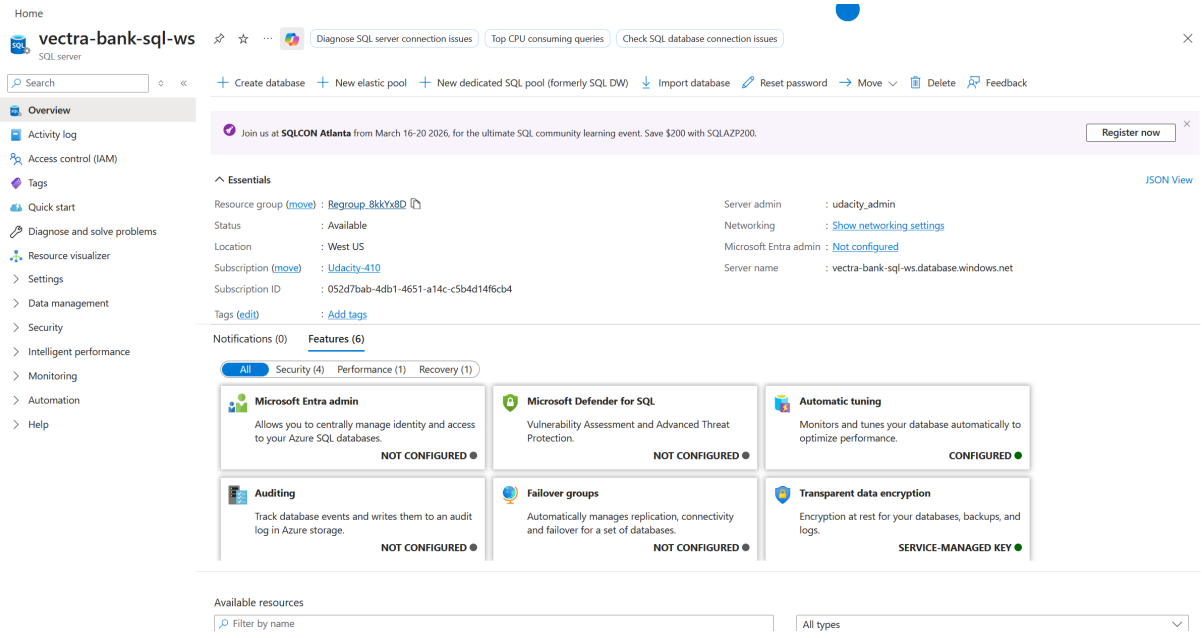

Figure 6: Azure SQL Server – Vectra Bank Workspace

Azure SQL – Firewall Rules

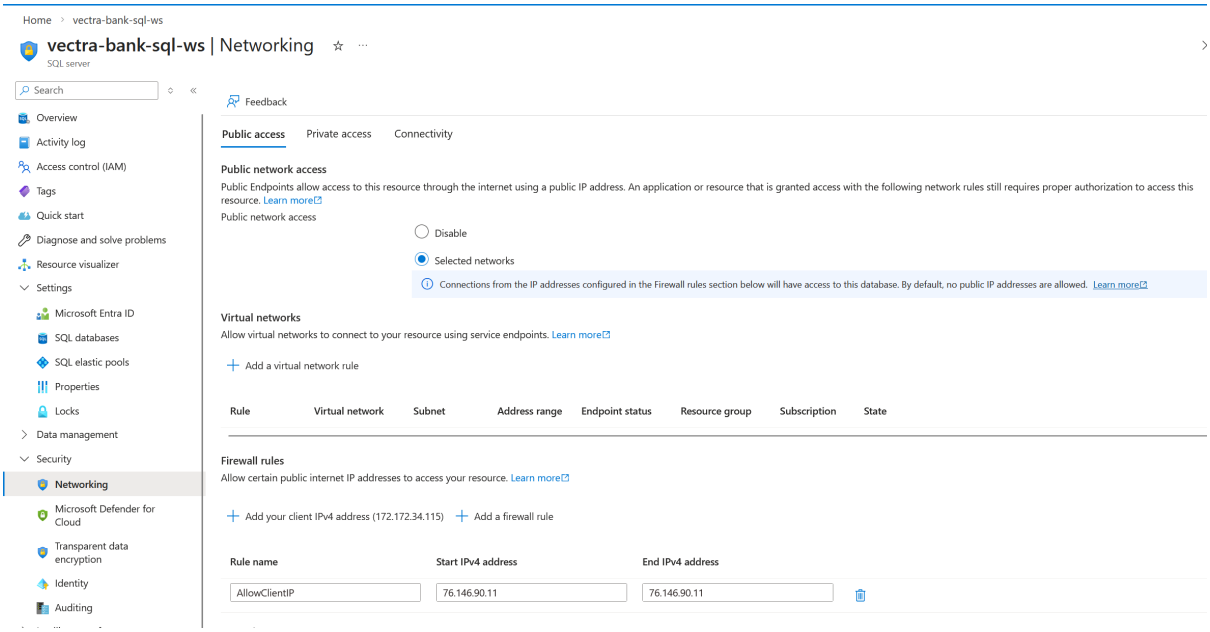

Figure 7: Azure SQL – Firewall and Networking Rules

Azure SQL – Query Editor

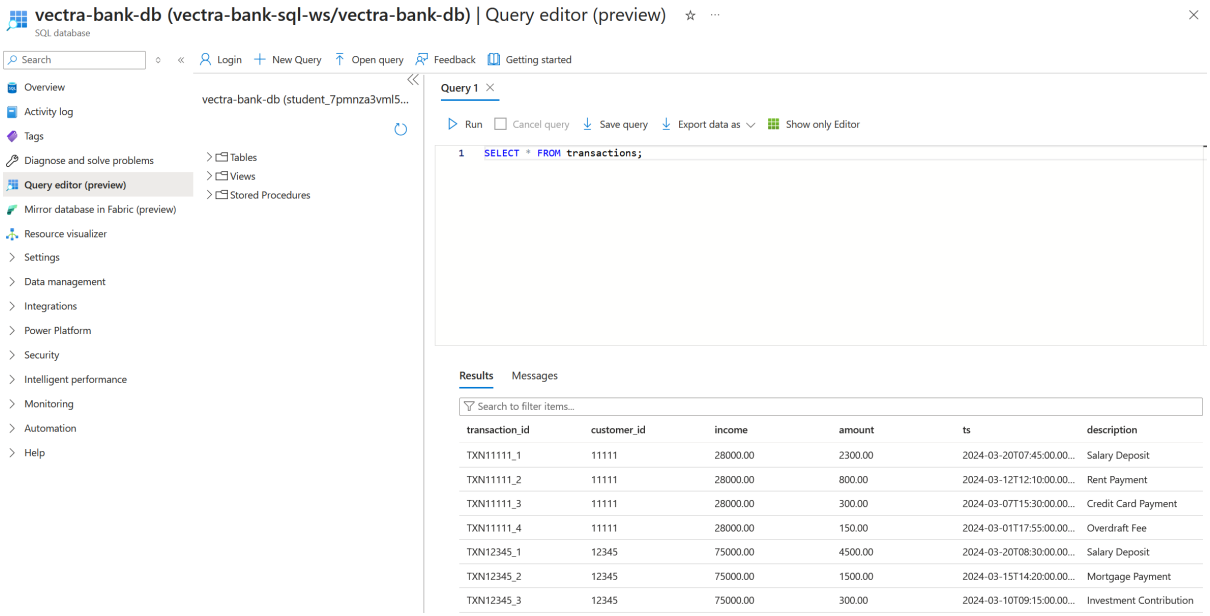

Figure 8: Azure SQL – Query Editor showing customer transactions

Azure SQL – Table Schema

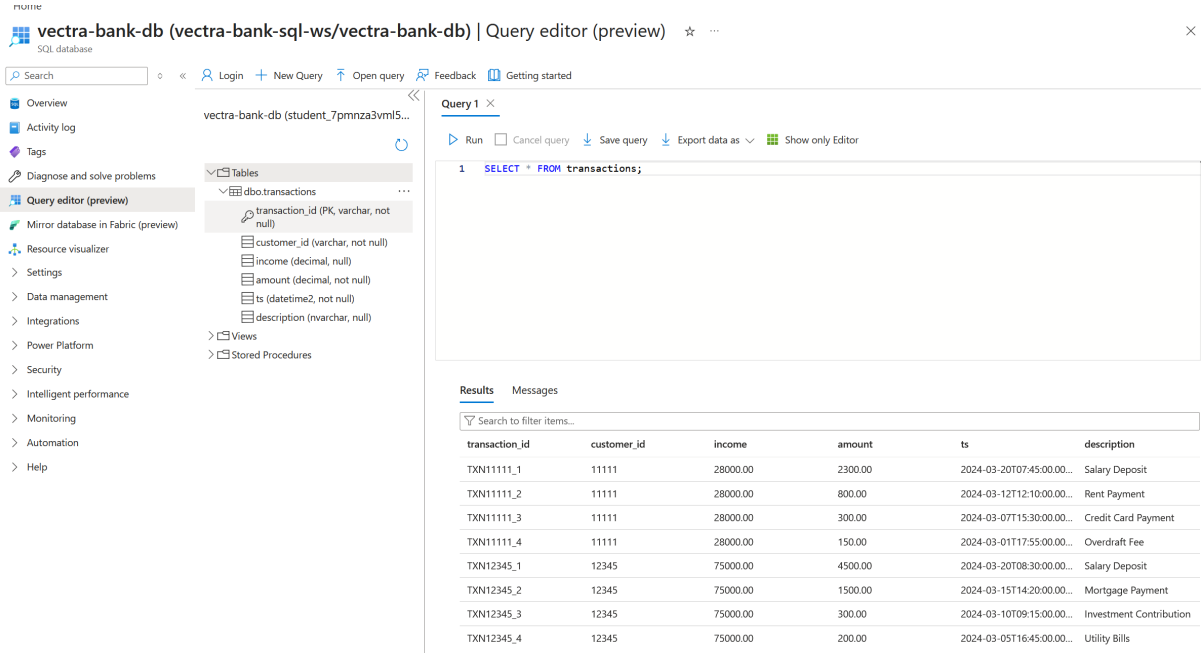

Figure 9: Azure SQL – Table Schema for customer_transactions

Appendix B: System Output Screenshots

Test Results (23/23 Pass)

```
======================================================================
  BANKING MULTI-AGENT RAG SYSTEM -- COMPONENT TESTS
======================================================================

  --- Pydantic Models ---
  [PASS] CustomerProfile creation
  [PASS] Credit score clamped to 850
  [PASS] EnhancedBankingReport creation
  [PASS] Risk score clamped to 100
  [PASS] Report display() works

  --- Blob Storage ---
  [PASS] BlobStorageConnector initialised
  [PASS] Sample documents uploaded (5)
  [PASS] Document content readable

  --- Azure Embedding Function ---
2026-02-21 13:49:31,464 - azure_embedding - INFO - Azure OpenAI embedding client initialised (deployment=text-embedding-ada-002).
  [PASS] Azure embedding function created
2026-02-21 13:49:32,037 - httpx - INFO - HTTP Request: POST https://final-project-udacity-ai.cognitiveservices.azure.com/openai/deployments/text-embedding-ada-002/embed
dings?api-version=2024-06-01 "HTTP/1.1 200 OK"
  [PASS] Embedding returned (dim=1536)

  --- ChromaDB ---
[OK] ChromaDB client initialized successfully
  [OK] Collection 'fraud_detection' initialized
  [OK] Collection 'loan_policies' initialized
  [OK] Collection 'customer_support' initialized
  [OK] Collection 'risk_assessment' initialized
  [OK] Collection 'transaction_monitoring' initialized
  [OK] Collection 'compliance' initialized
  [PASS] ChromaDBManager initialised
  [PASS] Collections available (6)
  [OK] Stored 1 chunks from test_doc.md in fraud_detection
  [PASS] Document chunked & stored (1 chunks)
  [PASS] Semantic search returned results (2)
```

Figure 10: Test Results – Part 1: Model, Blob, Embedding, ChromaDB, RAG tests

```
  --- DataConnector ---
2026-02-21 13:49:38,984 - data_connector - INFO - Azure SQL Database connection verified successfully.
  [PASS] DataConnector initialised
  [PASS] Azure SQL connection LIVE
  [PASS] fetch_income returned 75000.0
  [PASS] fetch_transactions returned 4 rows

  --- Semantic Kernel Agents ---
2026-02-21 13:49:41,794 - EnhancedBankingSequentialOrchestration - INFO - Initializing EnhancedBankingSequentialOrchestration ...
2026-02-21 13:49:42,160 - azure_embedding - INFO - Azure OpenAI embedding client initialised (deployment=text-embedding-ada-002).
2026-02-21 13:49:42,161 - EnhancedBankingSequentialOrchestration - INFO - Azure OpenAI embedding function available -- ChromaDB will use text-embedding-ada-002.
[OK] ChromaDB client initialized successfully
2026-02-21 13:49:42,254 - azure_embedding - INFO - Azure embedding credentials not provided -- ChromaDB will use its default embedding function.
  [OK] Collection 'fraud_detection' initialized (existing embeddings retained)
2026-02-21 13:49:42,368 - azure_embedding - INFO - Azure embedding credentials not provided -- ChromaDB will use its default embedding function.
  [OK] Collection 'loan_policies' initialized (existing embeddings retained)
2026-02-21 13:49:42,474 - azure_embedding - INFO - Azure embedding credentials not provided -- ChromaDB will use its default embedding function.
  [OK] Collection 'customer_support' initialized (existing embeddings retained)
2026-02-21 13:49:42,584 - azure_embedding - INFO - Azure embedding credentials not provided -- ChromaDB will use its default embedding function.
  [OK] Collection 'risk_assessment' initialized (existing embeddings retained)
2026-02-21 13:49:42,702 - azure_embedding - INFO - Azure embedding credentials not provided -- ChromaDB will use its default embedding function.
  [OK] Collection 'transaction_monitoring' initialized (existing embeddings retained)
2026-02-21 13:49:42,826 - azure_embedding - INFO - Azure embedding credentials not provided -- ChromaDB will use its default embedding function.
  [OK] Collection 'compliance' initialized (existing embeddings retained)
2026-02-21 13:49:44,000 - data_connector - INFO - Azure SQL Database connection verified successfully.
2026-02-21 13:49:44,001 - EnhancedBankingSequentialOrchestration - INFO - Azure SQL DataConnector is ONLINE.
2026-02-21 13:49:44,401 - EnhancedBankingSequentialOrchestration - INFO - Azure OpenAI chat service registered with Semantic Kernel.
2026-02-21 13:49:44,405 - EnhancedBankingSequentialOrchestration - INFO - Loaded 5 policy documents (3 categories).
  [PASS] Orchestration system initialised
2026-02-21 13:49:44,408 - EnhancedBankingSequentialOrchestration - INFO - Created 6 specialised banking agents.
  [PASS] Created 6 agents
  [PASS] All 6 agent names correct

  --- RAG Integration ---
  [PASS] Policy extraction works
  [PASS] Semantic kernel context generated

============================================================
  TESTS: 23/23 passed, 0/23 failed
============================================================
```

Figure 11: Test Results – Part 2: SQL, Orchestration tests and summary (23/23)

Demo Output (Single Scenario)

```
================================================================================
  ENHANCED BANKING ANALYSIS REPORT — enhanced_4d3e3b11
================================================================================
  Customer ID   : 12345
  Query         : I need comprehensive financial planning including investments and retirement options. Please also review my account for any suspicious activity and assess my lo
an eligibility.
  Generated     : 2026-02-21 13:52:59
  Generated By  : EnhancedBankingOrchestration
================================================================================

  EXECUTIVE SUMMARY
  ----------------------------------------
  ### Executive Report: XYZ Bank Risk and Operational Overview

#### **Executive Summary**
XYZ Bank faces heightened operational, credit, market, and compliance risks with an overall risk score categorized as high. Key stress points include an increase in non-performin
g loans (NPLs), market exposure to rising interest rates, and emerging cybersecurity vulnerabilities. Moreover, compliance lapses in Know Your Customer (KYC) reviews and Anti-Mon
ey Laundering (AML) monitoring have drawn attention fro

  RISK PROFILE
  ----------------------------------------
  Risk Assessment      : low
  Risk Score           : 12.0 / 100
  Fraud Risk Level     : low
  Loan Eligibility     : Yes
  Compliance Status    : compliant
  Financial Health     : 88.0 / 100

  KEY FINDINGS
  ----------------------------------------
  1. Analysis completed for customer 12345.
  2. Customer income of $75,000.00 places them in a strong financial position.
  3. Excellent credit score (780) — eligible for premium lending products.
  4. 4 recent transactions totalling $6,500.00 reviewed.
  5. Policy documents referenced from: fraud_detection.
  6. Agent 'Enhanced_Data_Gatherer' contributed to the analysis.
  7. Agent 'Enhanced_Fraud_Analyst' contributed to the analysis.
  8. Agent 'Enhanced_Loan_Analyst' contributed to the analysis.
  9. Agent 'Enhanced_Support_Specialist' contributed to the analysis.
  10. Agent 'Enhanced_Risk_Analyst' contributed to the analysis.
  11. Agent 'Enhanced_Synthesis_Coordinator' contributed to the analysis.
```

Figure 12: Demo Output – Part 1: Agent activations and processing

```
RECOMMENDATIONS
----------------------------------------
1. Maintain annual review cycle — low risk profile.

NEXT BEST ACTIONS
----------------------------------------
1. Send personalised analysis summary to customer.
2. Update CRM with latest risk assessment.
3. Pre-approve for premium product upgrade.
4. Schedule follow-up contact within 30 days.

AGENT ACTIVATIONS
----------------------------------------
[PASS] Enhanced_Data_Gatherer -- 6.15s
[PASS] Enhanced_Fraud_Analyst -- 12.55s
[PASS] Enhanced_Loan_Analyst -- 23.86s
[PASS] Enhanced_Support_Specialist -- 31.89s
[PASS] Enhanced_Risk_Analyst -- 42.45s
[PASS] Enhanced_Synthesis_Coordinator -- 49.94s

PROCESSING METRICS
----------------------------------------
total_processing_time_seconds: 61.2
agents_activated: 6
policies_referenced: 1
rag_results_returned: 2
risk_score: 12.0
customer_income: 75000.0
customer_credit_score: 780
================================================================================
```

Figure 13: Demo Output – Part 2: Final report with risk assessment

Evaluation Output (5/5 Scenarios Pass)

```
================================================================================
  ENHANCED BANKING ANALYSIS REPORT — enhanced_1e78f0d4
================================================================================
  Customer ID    : 67890
  Query          : What banking products would you recommend for someone in my financial situation?
  Generated      : 2026-02-21 13:59:36
  Generated By   : EnhancedBankingOrchestration
================================================================================

  EXECUTIVE SUMMARY
  ----------------------------------------
  Certainly! Drawing from comprehensive analyses across various aspects—data security, fraud prevention, loan portfolio analysis, customer support, and overall risk profile—I wil
l synthesise the findings into a concise, actionable banking report.

---

### Executive Summary
This report presents key findings and recommendations to address emerging risks and optimise banking operations. While the institution shows robust performance in loan management
 and support operations, there are identifiable v

  RISK PROFILE
  ----------------------------------------
  Risk Assessment      : high
  Risk Score           : 52.0 / 100
  Fraud Risk Level     : medium
  Loan Eligibility     : Yes
  Compliance Status    : compliant
  Financial Health     : 48.0 / 100

  KEY FINDINGS
  ----------------------------------------
  1. Analysis completed for customer 67890.
  2. 4 recent transactions totalling $4,950.00 reviewed.
  3. Policy documents referenced from: fraud_detection.
  4. Agent 'Enhanced_Data_Gatherer' contributed to the analysis.
  5. Agent 'Enhanced_Fraud_Analyst' contributed to the analysis.
  6. Agent 'Enhanced_Loan_Analyst' contributed to the analysis.
  7. Agent 'Enhanced_Support_Specialist' contributed to the analysis.
  8. Agent 'Enhanced_Risk_Analyst' contributed to the analysis.
  9. Agent 'Enhanced_Synthesis_Coordinator' contributed to the analysis.

  RECOMMENDATIONS
  ----------------------------------------
  1. Schedule quarterly risk reassessment.
  2. Enable enhanced transaction monitoring alerts.

  NEXT BEST ACTIONS
  ----------------------------------------
  1. Assign dedicated relationship manager within 24 hours.
  2. Run full AML/KYC compliance check.
  3. Send personalised analysis summary to customer.
  4. Update CRM with latest risk assessment.
  5. Schedule follow-up contact within 30 days.
```

Figure 14: Evaluation Output – Part 1: Scenarios processing

```
AGENT ACTIVATIONS
----------------------------------------
[PASS] Enhanced_Data_Gatherer -- 6.23s
[PASS] Enhanced_Fraud_Analyst -- 14.30s
[PASS] Enhanced_Loan_Analyst -- 20.74s
[PASS] Enhanced_Support_Specialist -- 32.50s
[PASS] Enhanced_Risk_Analyst -- 34.35s
[PASS] Enhanced_Synthesis_Coordinator -- 39.18s

PROCESSING METRICS
----------------------------------------
total_processing_time_seconds: 41.6
agents_activated: 6
policies_referenced: 1
rag_results_returned: 2
risk_score: 52.0
customer_income: 45000.0
customer_credit_score: 680
================================================================================

================================================================================
EVALUATION SUMMARY
================================================================================
[PASS] Scenario 1: Customer 12345 -- success  (risk=12.0, agents=6)
[PASS] Scenario 2: Customer 67890 -- success  (risk=52.0, agents=6)
[PASS] Scenario 3: Customer 11111 -- success  (risk=94.0, agents=6)
[PASS] Scenario 4: Customer 12345 -- success  (risk=12.0, agents=6)
[PASS] Scenario 5: Customer 67890 -- success  (risk=52.0, agents=6)
================================================================================

Summary saved to C:\Training\Udacity\Microsoft_Foundry\Projects\Project_4\reports\evaluation_summary.json
S C:\Training\Udacity\Microsoft_Foundry\Projects\Project_4>
```

Figure 15: Evaluation Output – Part 2: All 5 scenarios PASS summary

Report Sample (JSON)

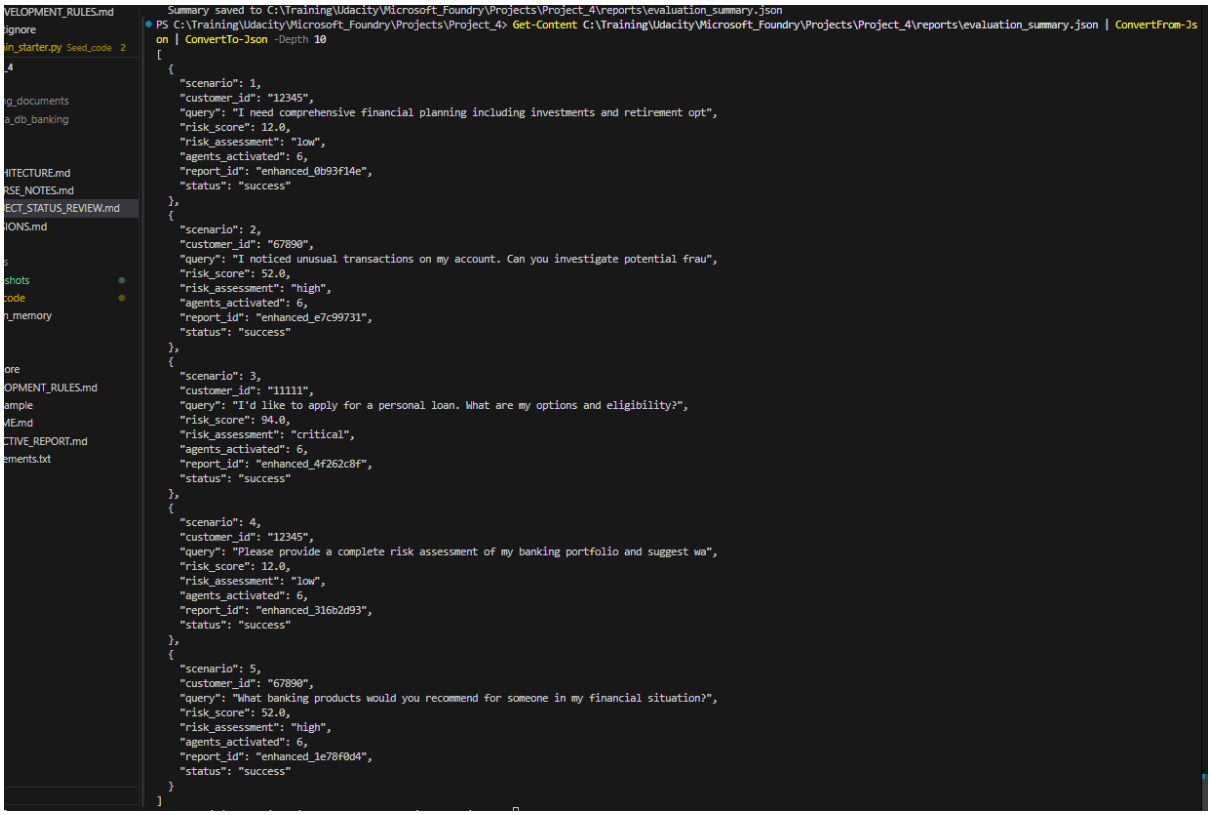

Figure 16: Evaluation Summary JSON – All 5 scenarios with risk scores